CS285 HW2：Policy Gradients
答案笔记、TODO 代码实现与实验分析

摘要

为什么 policy gradient 方差大，为什么 reward-to-go、advantage normalization、neural network baseline 和 GAE 能改善训练。

目录

1 作业整体任务与最快完成路线 **4**
- 1.1 这份作业到底要做什么 . . . 4
- 1.2 建议的完成顺序 . . . 4
- 1.3 一句话理解本作业主线 . . . 4

2 核心知识点总结 **5**
- 2.1 Policy Gradient 目标函数 . . . 5
- 2.2 为什么原始 Policy Gradient 方差大 . . . 5
- 2.3 Reward-to-go . . . 5
- 2.4 Advantage 与 baseline . . . 6
- 2.5 Advantage normalization . . . 6
- 2.6 GAE：Generalized Advantage Estimation . . . 6

3 TODO 代码实现与解释 **7**
- 3.1 采样 trajectory：`cs285/infrastructure/utils.py` . . . 7
- 3.2 训练循环：`cs285/scripts/run_hw2.py` . . . 8
- 3.3 策略网络：`cs285/networks/policies.py` . . . 9
  - 3.3.1 get_action . . . 9
  - 3.3.2 forward . . . 10
  - 3.3.3 actor update . . . 10
- 3.4 价值函数 baseline：`cs285/networks/critics.py` . . . 11
- 3.5 PGAgent 主逻辑：`cs285/agents/pg_agent.py` . . . 12

3.5.1 update 主函数 . . . 12
3.5.2 计算 Q values . . . 13
3.5.3 trajectory-centric return . . . 13
3.5.4 reward-to-go . . . 14
3.5.5 advantage 与 GAE . . . 14

4 实验命令与结果分析 17
4.1 画图说明 . . . 17
4.2 Experiment 1：CartPole . . . 17
4.2.1 运行命令 . . . 17
4.2.2 结果分析 . . . 18
4.2.3 问题回答 . . . 19
4.3 Experiment 2：HalfCheetah 与 Neural Network Baseline . . . 19
4.3.1 运行命令 . . . 19
4.3.2 结果分析 . . . 20
4.3.3 问题回答 . . . 21
4.4 Experiment 3：LunarLander 与 GAE . . . 21
4.4.1 运行命令 . . . 21
4.4.2 结果分析 . . . 21
4.4.3 问题回答 . . . 22
4.5 Experiment 4：InvertedPendulum 超参数调节与 sample efficiency . . . 22
4.5.1 运行命令 . . . 22
4.5.2 超参数表 . . . 23
4.5.3 结果分析 . . . 23
4.6 Extra Credit：Humanoid . . . 24

5 Section 8 理论题完整答案 25
5.1 题目设定 . . . 25
5.2 8.1 Applying policy gradients . . . 25
5.2.1 (a) 用 policy gradient 求 $\nabla_\theta J(\theta)$ . . . 25
5.2.2 (b) 直接计算 expected return 并验证 . . . 26
5.3 8.2 原始 policy gradient 的方差 . . . 26
5.4 8.3 Return-to-go 作为 advantage estimator . . . 26
5.4.1 (a) 写出 modified policy gradient 并说明 unbiased . . . 26
5.4.2 (b) Return-to-go 方差 . . . 27
5.5 8.4 Importance sampling policy gradients . . . 27
5.5.1 (a) 写出 importance sampling policy gradient . . . 28
5.5.2 (b) 计算方差并讨论 H 变大时的影响 . . . 28

6 Survey 时间估计 29

7 最终提交说明 **30**
7.1 报告 PDF 应包含什么 30
7.2 代码与日志打包 30

8 自己的思考与总结 **31**
8.1 Policy Gradient 的根本困难是方差 31
8.2 Reward-to-go 是最自然的 credit assignment 修正 31
8.3 Baseline 不是越复杂越好，而是要训练充分 31
8.4 GAE 的 λ 没有绝对最优 31
8.5 Sample efficiency 不能只看 iteration 31
8.6 最终一句话总结 31

1 作业整体任务与最快完成路线

1.1 这份作业到底要做什么

本作业围绕 **Policy Gradient** 展开。核心任务可以概括为一句话：

> 训练一个策略网络 $\pi_\theta(a|s)$，让它在环境中采样动作，并通过回报 $R(\tau)$ 更新参数；然后逐步加入 reward-to-go、advantage normalization、value baseline 和 GAE 来降低梯度估计的方差。

从代码角度看，需要补全的主要模块有：

1. 采样 trajectory：如何让 agent 和 environment 交互并保存 $(s_t, a_t, r_t, s_{t+1}, d_t)$；
2. policy 网络：如何根据 observation 输出动作分布；
3. actor 更新：如何用 $-\log \pi_\theta(a_t|s_t)A_t$ 作为 loss；
4. return estimator：如何计算 discounted return 和 reward-to-go；
5. critic/value baseline：如何训练 $V_\phi(s)$；
6. GAE：如何递推计算 A_t^{GAE}。

1.2 完成顺序

完成路线如下：

1. 先补代码，不要一开始就跑大实验；
2. 用 CartPole 跑 2 次 iteration 做 debug；
3. 依次跑 Experiment 1–4；
4. 用 TensorBoard event 文件画图；
5. 把代码实现、实验曲线、问题回答、理论推导写进 PDF；
6. 最后再打包 `submit.zip`。

1.3 本作业主线

阶段	做法	目的
Vanilla PG	整条轨迹一个总回报	最基本，但方差大
Reward-to-go	当前动作只看未来奖励	利用因果性，降低方差
Advantage normalization	标准化 advantage	控制梯度尺度，训练更稳
Neural network baseline	减去 $V_\phi(s)$	进一步降低方差
GAE	TD 残差递推	在 bias 和 variance 之间折中

2 核心知识点总结

2.1 Policy Gradient 目标函数

强化学习的目标是最大化期望回报：

$$J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta(\tau)}[R(\tau)]. \tag{1}$$

其中 τ 是一条 trajectory：

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s_{T-1}, a_{T-1}). \tag{2}$$

使用 log-derivative trick，可以得到：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(\tau) R(\tau)\right]. \tag{3}$$

因为环境动力学不依赖策略参数，所以：

$$\nabla_\theta \log \pi_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t). \tag{4}$$

因此最基本的 policy gradient 估计器是：

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) R(\tau_i). \tag{5}$$

2.2 为什么原始 Policy Gradient 方差大

原始估计器把同一条 trajectory 的总回报分配给每一个动作。问题是：

- 早期动作和后面奖励可能有关；
- 但后面动作不可能影响前面已经发生的奖励；
- 如果仍然把整条轨迹总回报给所有动作，就会引入额外噪声。

直观来说，原始 PG 像是在说：

一场考试考砸了，于是把第一题、第二题、最后一题都平均背锅。

但是合理做法应该是：

第 t 步动作只对第 t 步之后的结果负责。

这就是 reward-to-go 的思想。

2.3 Reward-to-go

Reward-to-go 写作：

$$Q_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}. \tag{6}$$

它只计算当前时刻之后的折扣回报。因此 policy gradient 变成：

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_{i,t}|s_{i,t}) Q_{i,t}. \tag{7}$$

2.4 Advantage 与 baseline

如果直接用 Q_t，尺度可能很大，方差也高。我们可以减去 baseline：

$$A_t = Q_t - b(s_t). \tag{8}$$

在本作业中，baseline 是神经网络 $V_\phi(s_t)$：

$$A_t = Q_t - V_\phi(s_t). \tag{9}$$

baseline 不改变梯度期望，因此不会引入偏差；它主要用来减少方差。

2.5 Advantage normalization

Advantage normalization 是把一个 batch 内的 advantage 做标准化：

$$\hat{A}_t = \frac{A_t - \mu_A}{\sigma_A + \epsilon}. \tag{10}$$

它的作用是让梯度尺度更稳定，避免更新一步太大、一步太小。

2.6 GAE：Generalized Advantage Estimation

GAE 从 TD residual 开始：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t). \tag{11}$$

递推形式为：

$$A_t^{\mathrm{GAE}} = \delta_t + \gamma\lambda A_{t+1}^{\mathrm{GAE}}. \tag{12}$$

其中 λ 控制 bias-variance tradeoff：

- $\lambda = 0$：接近 one-step TD，低方差、高偏差；
- $\lambda = 1$：接近 Monte Carlo return，低偏差、高方差；
- $0 < \lambda < 1$：折中。

3 TODO 代码实现与解释

3.1 采样 trajectory：cs285/infrastructure/utils.py

这一部分负责让 policy 和环境交互，收集一条 trajectory。

Listing 1: sample_trajectory 的核心实现

```python

def sample_trajectory(
    env: gym.Env,                           # 当前要交互的 Gym 环境，例如 CartPole/HalfCheetah
    policy: MLPPolicy,                      # 当前策略网络，用来根据 observation 采样 action
    max_length: int,                        # 单条 trajectory 允许的最大长度，防止 episode 无限跑下去
    render: bool = False,                   # 是否保存渲染图片；默认不保存，避免日志太大
) -> Dict[str, np.ndarray]:
    """Sample a rollout in the environment from a policy."""

    reset_out = env.reset()                 # 重置环境，得到初始 observation；不同 Gym 版本返回格式可能不同
    ob = reset_out[0] if isinstance(reset_out, tuple) else reset_out  # 兼容 Gym 0.26+ 的 (obs, info) 返回

    obs, acs, rewards, next_obs, terminals, image_obs = [], [], [], [], [], []  # 建立列表，逐步保存一条轨迹
    steps = 0                               # 记录当前 trajectory 已经走了多少步

    while True:                             # 不断和环境交互，直到 episode 结束或达到 max_length
        ac: np.ndarray = policy.get_action(ob)  # 用当前 observation 让 policy 采样一个动作

        step_out = env.step(ac)             # 把动作送入环境，得到下一状态、奖励、终止标志等信息
        if len(step_out) == 5:              # Gym 0.26+ 返回 obs, reward, terminated, truncated, info
            next_ob, rew, terminated, truncated, _ = step_out  # 拆出新状态、奖励和两种终止信号
            done = terminated or truncated  # terminated 是任务自然结束，truncated 是达到时间限制
        else:                               # 老版本 Gym 返回 obs, reward, done, info
            next_ob, rew, done, _ = step_out  # 直接使用 done 表示 episode 是否结束

        steps += 1                          # 环境交互步数加 1
        rollout_done: bool = done or (steps >= max_length)  # 只要环境结束或超过最大长度，就停止本条轨迹
```

```
28
29          obs.append(ob)                      # 保存当前状态 s_t
30          acs.append(ac)                      # 保存当前动作 a_t
31          rewards.append(rew)                 # 保存当前奖励 r_t
32          next_obs.append(next_ob)            # 保存下一状态 s_{t+1}
33          terminals.append(rollout_done)      # 保存这一时刻是否为 trajectory 结束点
34
35          ob = next_ob                        # 把下一状态变成下一轮循环的当前状态
36
37          if rollout_done:                    # 如果本条 trajectory 已经结束，就跳出 while
        循环
38              break
39
40      return {                                # 把收集到的列表转成 numpy array，供后续训练使
        用
41          "observation": np.array(obs, dtype=np.float32),       # 所有状态 s_t
42          "image_obs": np.array(image_obs, dtype=np.uint8),     # 渲染图像；默认通常为空
43          "reward": np.array(rewards, dtype=np.float32),        # 所有奖励 r_t
44          "action": np.array(acs, dtype=np.float32),            # 所有动作 a_t
45          "next_observation": np.array(next_obs, dtype=np.float32),  # 所有下一状态 s_{t
        +1}
46          "terminal": np.array(terminals, dtype=np.float32),    # episode 结束标志
47      }
```

解释。 这个函数做的事情是：

1. 用 `env.reset()` 得到初始状态；
2. 用 `policy.get_action(ob)` 根据当前状态采样动作；
3. 用 `env.step(ac)` 推进环境；
4. 保存 observation、action、reward、next observation、terminal；
5. 如果 episode 结束或达到最大长度，就停止。

特别注意：Gym 不同版本返回值不同，有的 `step` 返回 4 个量，有的返回 5 个量。因此代码里兼容了 `terminated` 和 `truncated`。

3.2 训练循环：`cs285/scripts/run_hw2.py`

核心是采样一批轨迹，然后交给 agent 更新。

Listing 2: run_hw2.py 中采样 batch 的 TODO

```
1
2  trajs, envsteps_this_batch = utils.sample_trajectories(  # 采样一批 trajectories，并返
       回本批消耗的环境步数
3      env,                                # 当前训练环境
```

```
    agent.actor,                    # 用 agent 的 actor policy 来和环境交互
    args.batch_size,                # 本轮至少要采集的 state-action 样本数
    max_ep_len,                     # 单个 episode 的最大长度
)

total_envsteps += envsteps_this_batch # 累加总环境交互步数，用作所有图的横坐标
```

Listing 3: run_hw2.py 中更新 agent 的 TODO

```

train_info: dict = agent.update(    # 用刚采样到的一批数据更新 policy/critic
    trajs_dict["observation"],      # 传入 batch 中所有 observation
    trajs_dict["action"],           # 传入 batch 中所有 action
    trajs_dict["reward"],           # 传入每条 trajectory 的 reward 序列
    trajs_dict["terminal"],         # 传入 terminal 标志，用于处理 episode 边界
)
```

解释。 每一次 policy gradient iteration 都要重新采样一批新的数据。这一点和 supervised learning 不同：

- supervised learning 的数据集通常固定；
- policy gradient 每更新一次策略，数据分布就变了，所以需要重新采样。

3.3 策略网络：cs285/networks/policies.py

3.3.1 get_action

Listing 4: 根据 observation 采样动作

```

@torch.no_grad()                    # 采样动作时不需要反向传播，关闭梯度可节省显存和时间
def get_action(self, obs: np.ndarray) -> np.ndarray:
    """Takes a single observation and returns a single action."""

    if obs.ndim == 1:               # 如果输入是单个 observation，形状通常是一维
        obs = obs[None]             # 增加 batch 维度，变成 shape = (1, obs_dim)

    obs_t = ptu.from_numpy(obs)     # 把 numpy array 转成 torch tensor，并放到正确 device 上
    action_distribution = self.forward(obs_t)  # 前向传播，得到动作概率分布
    action = action_distribution.sample()      # 从动作分布中采样动作，保留探索性

    return ptu.to_numpy(action)[0]  # 转回 numpy，并去掉 batch 维度，返回单个动作
```

解释。 这里不是输出一个固定动作，而是输出动作分布后再采样。原因是 policy gradient 需要探索。如果每次都选最大概率动作，agent 很容易陷入局部最优。

3.3.2 forward

Listing 5: 离散动作与连续动作的分布输出

```

def forward(self, obs: torch.FloatTensor):
    if self.discrete:                           # 离散动作空间，例如 CartPole 的 left/right
        logits = self.logits_net(obs)           # 网络输出每个离散动作的 logits
        return distributions.Categorical(logits=logits)  # 用 logits 构造 categorical 动作分布
    else:                                       # 连续动作空间，例如 HalfCheetah 的关节力矩
        mean = self.mean_net(obs)               # 网络输出高斯分布的均值 mu(s)
        std = torch.exp(self.logstd)            # logstd 是可学习参数，取 exp 后得到标准差 sigma
        normal_dist = distributions.Normal(mean, std)    # 对每个动作维度构造独立高斯分布
        return distributions.Independent(normal_dist, 1)  # 把多维高斯看成一个联合动作分布
```

解释。 离散动作环境，例如 CartPole，策略输出 Categorical distribution；连续动作环境，例如 HalfCheetah，策略输出 Gaussian distribution。

3.3.3 actor update

Listing 6: Policy gradient actor loss

```

def update(
    self,
    obs: np.ndarray,                    # 一个 batch 的 observation
    actions: np.ndarray,                # 这些 observation 下实际采样过的 action
    advantages: np.ndarray,             # 每个 action 对应的 advantage 权重
) -> dict:
    """Implements the policy gradient actor update."""

    obs = ptu.from_numpy(obs)           # observation 从 numpy 转为 torch tensor
    actions = ptu.from_numpy(actions) # action 从 numpy 转为 torch tensor
    advantages = ptu.from_numpy(advantages)  # advantage 从 numpy 转为 torch tensor

    if self.discrete:                   # 如果是离散动作空间
        actions = actions.long()        # Categorical.log_prob 要求 action 是整数类别索引
```

```
16
17      action_distribution = self.forward(obs)       # 重新计算当前 policy 下的动作分布
        pi_theta(a|s)
18      log_probs = action_distribution.log_prob(actions)  # 取出采样动作的 log probability
19
20      loss = -(log_probs * advantages).mean()       # PyTorch 最小化 loss，所以加负号实现
        梯度上升
21
22      self.optimizer.zero_grad()          # 清空上一轮累积的梯度
23      loss.backward()                     # 对 actor loss 做反向传播，计算参数梯度
24      self.optimizer.step()               # 用 optimizer 更新 policy 网络参数
25
26      return {                            # 返回日志信息，供 TensorBoard 记录
27          "Actor Loss": ptu.to_numpy(loss),  # 把 loss 转成 numpy/标量方便记录
28      }
```

解释。 Policy gradient 希望最大化：

$$\mathbb{E}\left[\log \pi_\theta(a_t|s_t) A_t\right]. \tag{13}$$

但是 PyTorch optimizer 默认最小化 loss，所以写成：

$$\mathcal{L}_{actor} = -\mathbb{E}\left[\log \pi_\theta(a_t|s_t) A_t\right]. \tag{14}$$

如果 $A_t > 0$，说明这个动作比平均水平好，更新会增加该动作概率；如果 $A_t < 0$，说明这个动作不好，更新会降低该动作概率。

3.4 价值函数 baseline：cs285/networks/critics.py

Listing 7: Value critic 的 forward 和 update

```
1
2   def forward(self, obs: torch.Tensor) -> torch.Tensor:
3       value = self.network(obs)           # value network 输入状态 s，输出 V_phi(s)
4       return value.squeeze(-1)            # 去掉最后一维，使形状从 (B, 1) 变成 (B,)
5
6
7   def update(self, obs: np.ndarray, q_values: np.ndarray) -> dict:
8       obs = ptu.from_numpy(obs)           # 把 observation 转成 torch tensor
9       q_values = ptu.from_numpy(q_values)  # 把 Monte Carlo return/reward-to-go 转成
        tensor，作为监督目标
10
11      values = self.forward(obs)          # 当前 critic 对每个状态的价值预测 V_phi(s)
12      loss = F.mse_loss(values, q_values)  # 用 MSE 让 V_phi(s) 拟合 q_values
13
14      self.optimizer.zero_grad()          # 清空 critic 旧梯度
15      loss.backward()                     # 反向传播，计算 critic 参数梯度
```

```
        self.optimizer.step()              # 更新 critic/value network

        return {                           # 返回 baseline loss 供日志记录
            "Baseline Loss": ptu.to_numpy(loss),  # 记录 critic 的拟合误差
        }
```

解释。 Critic 的目标是拟合 reward-to-go：

$$V_\phi(s_t) \approx Q_t. \tag{15}$$

所以用均方误差：

$$\mathcal{L}_{critic} = \frac{1}{B}\sum_t (V_\phi(s_t) - Q_t)^2. \tag{16}$$

3.5 PGAgent 主逻辑：cs285/agents/pg_agent.py

3.5.1 update 主函数

Listing 8: PGAgent.update 的完整逻辑

```

    def update(
        self,
        obs: Sequence[np.ndarray],        # 多条 trajectory 的 observation 列表
        actions: Sequence[np.ndarray],    # 多条 trajectory 的 action 列表
        rewards: Sequence[np.ndarray],    # 多条 trajectory 的 reward 列表
        terminals: Sequence[np.ndarray],  # 多条 trajectory 的 terminal 标志列表
    ) -> dict:

        q_values: Sequence[np.ndarray] = self._calculate_q_vals(rewards)  # 先根据 reward 计算每个时间步的 Q 值估计

        obs = np.concatenate(obs)          # 把多条 trajectory 的 observation 拼成一个 batch
        actions = np.concatenate(actions) # 把多条 trajectory 的 action 拼成一个 batch
        rewards = np.concatenate(rewards) # 把 reward 也拉平成一个 batch，GAE 会用到
        terminals = np.concatenate(terminals)  # 拉平 terminal，用来判断 episode 边界
        q_values = np.concatenate(q_values)    # 拉平 Q values，使其与 obs/actions 一一对应

        advantages: np.ndarray = self._estimate_advantage(  # 计算 policy gradient 需要的 advantage
            obs,                           # 所有状态
            rewards,                       # 所有奖励
            q_values,                      # Monte Carlo Q 值估计
            terminals,                     # episode 结束标志
        )

        info: dict = self.actor.update(obs, actions, advantages)  # 用 advantage 加权的 log_prob 更新 actor
```

```
26
27      if self.critic is not None:          # 如果启用了 neural network baseline，就同时训练
        critic
28          for _ in range(self.baseline_gradient_steps):  # critic 每轮可以训练多步，提高 V
        (s) 拟合质量
29              critic_info: dict = self.critic.update(obs, q_values)  # 用 q_values 监督训
        练 value baseline
30          info.update(critic_info)         # 把 critic 的日志信息合并到返回字典中
31
32      return info                          # 返回 actor/critic 的训练日志
```

解释。 整个 agent update 的流程是：

1. 根据 rewards 计算 Q_t；
2. 把多条 trajectory 拼成一个 batch；
3. 计算 advantage；
4. 更新 actor；
5. 如果使用 baseline，再更新 critic。

3.5.2 计算 Q values

Listing 9: 选择 discounted return 或 reward-to-go

```
1
2  def _calculate_q_vals(self, rewards: Sequence[np.ndarray]) -> Sequence[np.ndarray]:
3      """Monte Carlo estimation of the Q function."""
4
5      if not self.use_reward_to_go:        # 如果没有开启 reward-to-go，就使用整条轨迹的总
       回报
6          q_values = [self._discounted_return(reward) for reward in rewards]  # 每条
       trajectory 内所有时间步共享同一个 return
7      else:                                # 如果开启 reward-to-go，就每个时间步只看自己之
       后的奖励
8          q_values = [self._discounted_reward_to_go(reward) for reward in rewards]  # 分
       别计算每条 trajectory 的 reward-to-go
9
10     return q_values                      # 返回 list，后面再 concatenate 成一个 batch
```

3.5.3 trajectory-centric return

Listing 10: 整条 trajectory 使用同一个 discounted return

```
1
```

```
def _discounted_return(self, rewards: Sequence[float]) -> Sequence[float]:
    """Full trajectory discounted return."""

    discounted_return = 0.0             # 用来累加整条 trajectory 的折扣总回报

    for t, reward in enumerate(rewards):  # 从 t=0 到 T-1 遍历奖励序列
        discounted_return += (self.gamma ** t) * reward  # 按 gamma^t 对每个 reward 做折扣后累加

    return np.array([discounted_return] * len(rewards), dtype=np.float32)  # 每个时间步都分配同一个总回报
```

解释。 这个版本中，一整条 trajectory 的每个 action 都使用相同的总回报。它简单，但方差高。

3.5.4 reward-to-go

Listing 11: 从后往前计算 reward-to-go

```

def _discounted_reward_to_go(self, rewards: Sequence[float]) -> Sequence[float]:
    """Reward-to-go."""

    q_values = np.zeros_like(rewards, dtype=np.float32)  # 创建与 rewards 同长度的数组，存放每个 t 的 Q_t
    running_sum = 0.0                   # 从后往前累加的临时变量，表示未来折扣回报

    for t in reversed(range(len(rewards))):  # 从最后一个时间步倒着往前算
        running_sum = rewards[t] + self.gamma * running_sum  # Q_t = r_t + gamma * Q_{t+1}
        q_values[t] = running_sum       # 把当前时间步的 reward-to-go 存入数组

    return q_values                     # 返回每个时间步各自的未来折扣回报
```

解释。 假设 rewards 是 $[r_0, r_1, r_2]$，则返回：

$$\left[r_0 + \gamma r_1 + \gamma^2 r_2, \quad r_1 + \gamma r_2, \quad r_2\right]. \tag{17}$$

注意：这段代码必须放在 Python 文件里面，不能直接粘到 Bash 终端中。终端中直接输入 `q_values = np.zeros_like(...)` 会被 Bash 当成命令，从而出现 `syntax error near unexpected token`。

3.5.5 advantage 与 GAE

Listing 12: Advantage estimate 与 GAE 实现

```

```

```python
def _estimate_advantage(
    self,
    obs: np.ndarray,                    # 拉平后的 observation batch
    rewards: np.ndarray,                # 拉平后的 reward batch
    q_values: np.ndarray,               # Monte Carlo return 或 reward-to-go
    terminals: np.ndarray,              # 每个时间步是否为 episode 结束点
) -> np.ndarray:
    """Computes advantages."""

    if self.critic is None:             # 如果没有 baseline/critic
        advantages = q_values.copy()    # advantage 直接等于 Q 值估计
    else:                               # 如果有 neural network baseline
        obs_t = ptu.from_numpy(obs)     # 把 observation 转成 torch tensor
        values = ptu.to_numpy(self.critic(obs_t))  # 用 critic 预测每个状态的 V_phi(s)

        assert values.shape == q_values.shape  # 确保 V(s) 和 Q(s,a) 一一对应，避免广播错误

        if self.gae_lambda is None:    # 如果不使用 GAE
            advantages = q_values - values  # A_t = Q_t - V(s_t)
        else:                          # 如果使用 GAE-lambda
            batch_size = obs.shape[0] # batch 中总时间步数

            values = np.append(values, [0])       # 在最后补一个 0，方便访问 values[i+1]
            advantages = np.zeros(batch_size + 1) # 多开一位，方便递推 A_{t+1}

            for i in reversed(range(batch_size)): # 从后往前递推 GAE
                next_nonterminal = 1.0 - terminals[i]  # terminal 后不能继续 bootstrap 下一状态价值

                delta = (                   # TD residual: delta_t = r_t + gamma V(s_{t+1}) - V(s_t)
                    rewards[i]              # 当前即时奖励 r_t
                    + self.gamma * values[i + 1] * next_nonterminal  # 如果未终止，加入下一状态价值
                    - values[i]             # 减去当前状态价值 V(s_t)
                )

                advantages[i] = (           # GAE 递推: A_t = delta_t + gamma lambda A_{t+1}
                    delta                    # 当前一步 TD residual
                    + self.gamma            # discount factor gamma
                    * self.gae_lambda        # GAE 的 lambda，控制 bias-variance tradeoff
                    * advantages[i + 1]      # 下一时间步的 advantage
                    * next_nonterminal       # 如果 episode 结束，就切断递推
```

```
                )

            advantages = advantages[:-1]      # 去掉为了递推多开的最后一位

    if self.normalize_advantages:             # 如果开启 advantage normalization
        advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)  # 标准化成均值 0、方差约 1

    return advantages                         # 返回 actor update 使用的 advantage 权重
```

解释。 如果没有 critic，advantage 就直接等于 Q_t。如果有 critic 但不用 GAE，就用：

$$A_t = Q_t - V_\phi(s_t). \tag{18}$$

如果使用 GAE，就先计算：

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \tag{19}$$

再从后往前递推：

$$A_t = \delta_t + \gamma\lambda A_{t+1}. \tag{20}$$

其中 `next_nonterminal` 用来处理 episode 结束的情况：如果当前步已经 terminal，就不能继续使用下一个状态的价值。

4 实验命令与结果分析

4.1 画图说明

本节图像对应本次运行结果。图的横坐标统一使用 `Train_EnvstepsSoFar`，也就是 environment steps，而不是 policy gradient iterations。这样才能公平比较不同 batch size 的 sample efficiency。

4.2 Experiment 1：CartPole

4.2.1 运行命令

Listing 13: CartPole small batch 与 large batch 实验命令

```
python cs285/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 \
  --exp_name cartpole

python cs285/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 \
  -rtg --exp_name cartpole_rtg

python cs285/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 \
  -na --exp_name cartpole_na

python cs285/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 \
  -rtg -na --exp_name cartpole_rtg_na

python cs285/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 4000 \
  --exp_name cartpole_lb

python cs285/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 4000 \
  -rtg --exp_name cartpole_lb_rtg

python cs285/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 4000 \
  -na --exp_name cartpole_lb_na

python cs285/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 4000 \
  -rtg -na --exp_name cartpole_lb_rtg_na
```

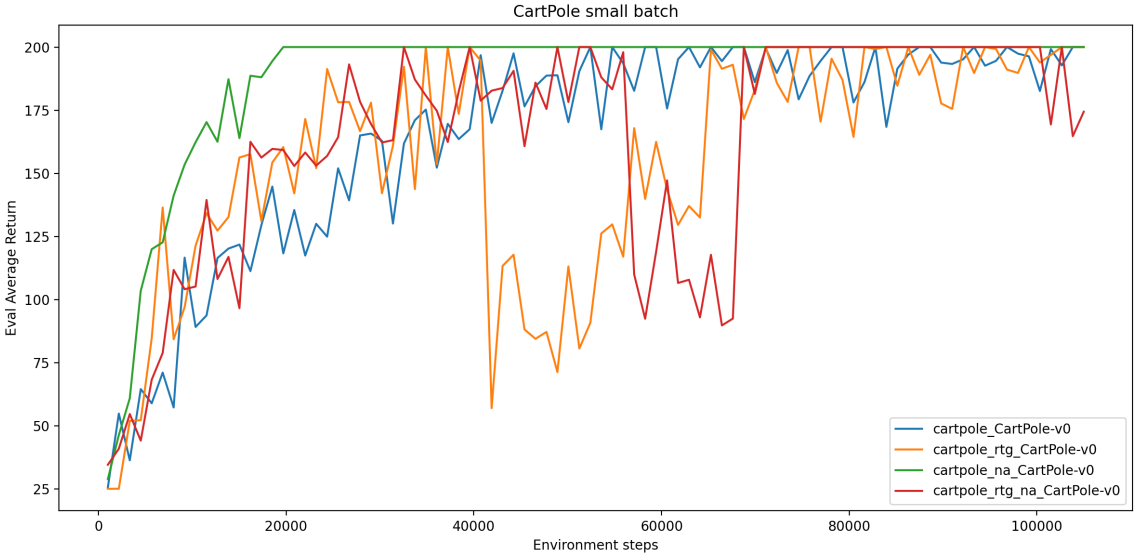


图 1: CartPole small batch：不同 reward estimator 与 advantage normalization 的学习曲线。

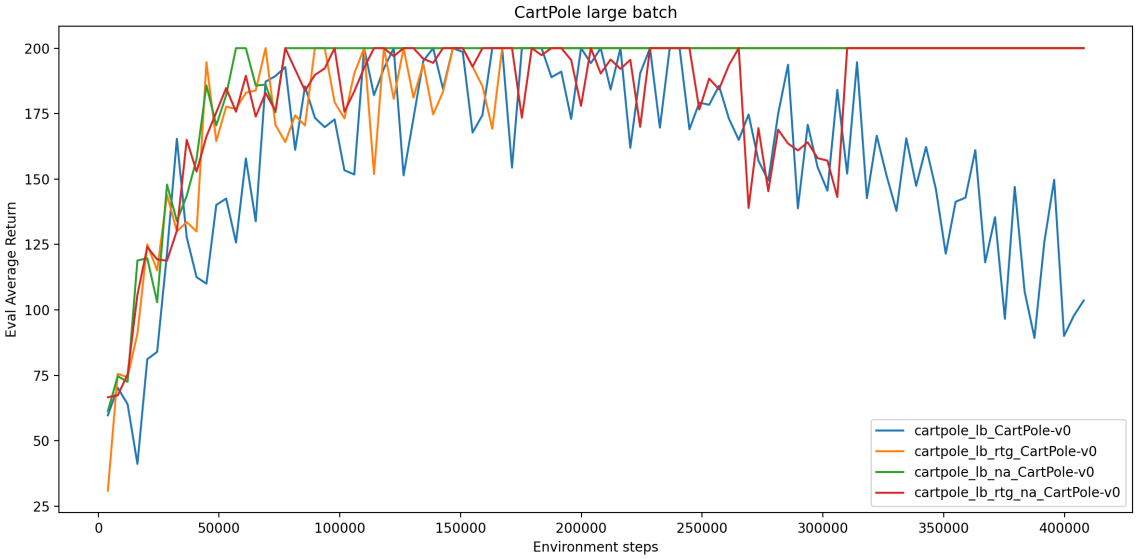


图 2: CartPole large batch：不同 reward estimator 与 advantage normalization 的学习曲线。

4.2.2 结果分析

从图 1 可以看到，在 small batch 设置下，四种方法最终基本都能接近或达到 CartPole 的最高分 200。其中 advantage normalization 的作用很明显：`cartpole_na` 收敛较快并且较稳定。`cartpole_rtg` 和 `cartpole_rtg_na` 也能达到高分，但 small batch 下曲线仍有波动，说明采样方差比较大。

从图 2 可以看到，大多数 large batch 方法也能达到 200。large batch 的曲线在使用 normalization 后更稳定，因为每次更新使用了更多样本，梯度估计方差更低。不过原始 `cartpole_lb` 在后期明显下降，说明仅仅增大 batch size 并不能完全解决 vanilla policy gradient 的高方差问题。

4.2.3 问题回答

1. 不使用 advantage normalization 时，trajectory-centric estimator 和 reward-to-go 哪个更好？ Reward-to-go 更好。原因是 reward-to-go 只把当前动作之后的奖励分配给当前动作，符合因果性；trajectory-centric estimator 把整条轨迹的回报分配给每一个动作，包括那些不可能影响过去奖励的动作，因此方差更大。

2. Advantage normalization 有没有帮助？ 有帮助。Advantage normalization 把一个 batch 中的 advantage 调整到零均值、单位方差，使梯度尺度更稳定。实验中 normalization 通常能带来更快、更平稳的学习曲线。

3. Batch size 有什么影响？ Batch size 增大后，每次更新的梯度估计更稳定，所以曲线往往更平滑；但是从 environment steps 的角度看，大 batch 每次更新消耗更多样本，不一定更 sample efficient。因此大 batch 能降低噪声，但不能替代 reward-to-go 或 advantage normalization。

4.3 Experiment 2：HalfCheetah 与 Neural Network Baseline

4.3.1 运行命令

Listing 14: HalfCheetah baseline 实验命令

```
# No baseline
python cs285/scripts/run_hw2.py --env_name HalfCheetah-v4 \
  -n 100 -b 5000 -rtg --discount 0.95 -lr 0.01 \
  --exp_name cheetah

# Baseline, baseline_gradient_steps = 5
python cs285/scripts/run_hw2.py --env_name HalfCheetah-v4 \
  -n 100 -b 5000 -rtg --discount 0.95 -lr 0.01 \
  --use_baseline -blr 0.01 -bgs 5 \
  --exp_name cheetah_baseline

# Baseline, baseline_gradient_steps = 1
python cs285/scripts/run_hw2.py --env_name HalfCheetah-v4 \
  -n 100 -b 5000 -rtg --discount 0.95 -lr 0.01 \
  --use_baseline -blr 0.01 -bgs 1 \
  --exp_name cheetah_baseline_bgs1
```

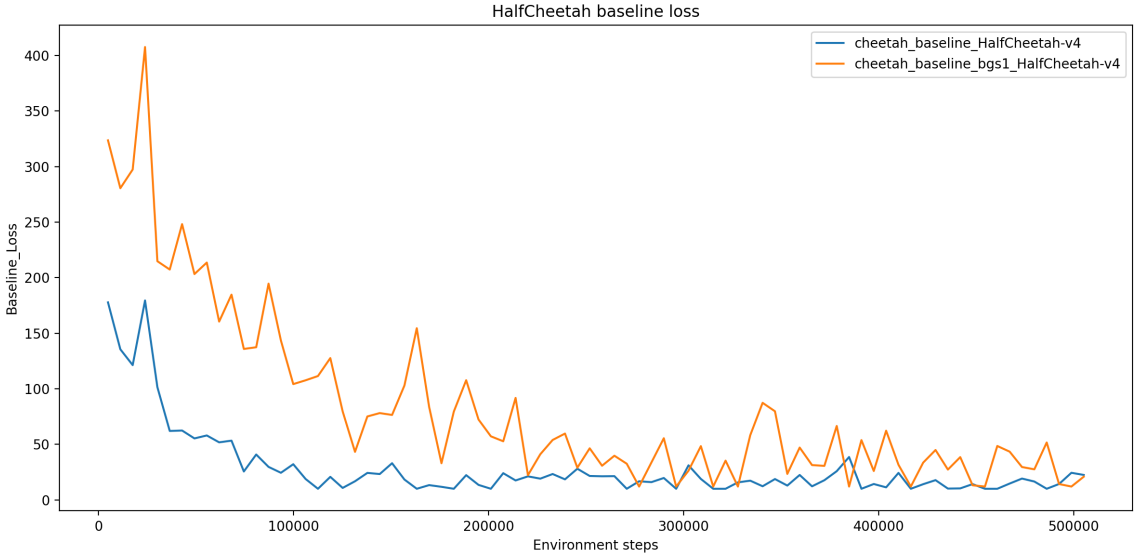


图 3: HalfCheetah baseline loss：比较 baseline gradient steps 为 5 和 1 的情况。

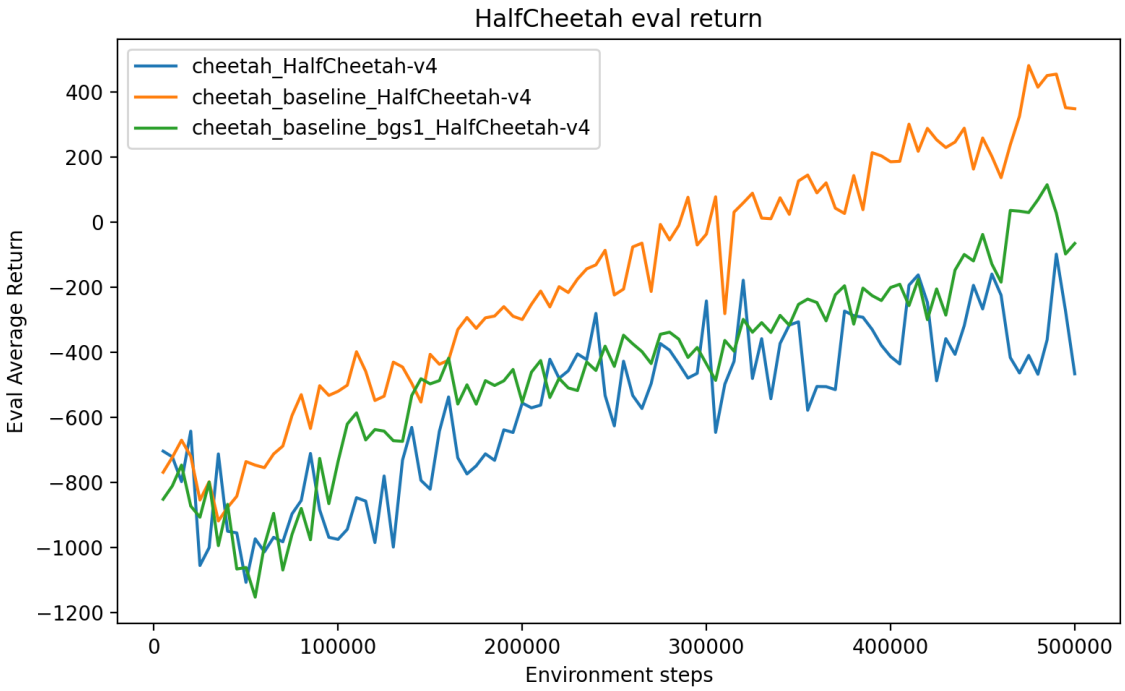


图 4: HalfCheetah evaluation return：无 baseline、baseline bgs=5、baseline bgs=1 的性能比较。

4.3.2 结果分析

图 3 显示，`bgs=5` 时 baseline loss 下降更快且更稳定；`bgs=1` 时 baseline loss 一开始明显更高，后期虽然也下降，但整体更不稳定。这说明只训练一步 baseline 不足以很好拟合 $V(s)$。

图 4 显示，加入 neural network baseline 后，HalfCheetah 的表现显著提升。无 baseline 时，evaluation return 长时间在负回报附近波动；使用 `bgs=5` 的 baseline 后，return 持续上升，最终超过 300，接近 400–500；使用 `bgs=1` 时虽然比无 baseline 好，但明显不如 `bgs=5`。

4.3.3 问题回答

Baseline 的作用是拟合 $V_\phi(s)$，然后用：

$$A_t = Q_t - V_\phi(s_t) \tag{21}$$

作为 policy gradient 的权重。一个训练充分的 baseline 能降低 advantage 的方差，使策略更新更稳定。降低 baseline gradient steps 后，value network 训练不充分，baseline loss 更高，advantage estimate 更不准确，因此 policy performance 也更差。

4.4 Experiment 3：LunarLander 与 GAE

4.4.1 运行命令

Listing 15: LunarLander 不同 GAE lambda 的实验命令

```
for lam in 0 0.95 0.98 0.99 1; do
  python cs285/scripts/run_hw2.py \
    --env_name LunarLander-v2 --ep_len 1000 \
    --discount 0.99 -n 300 -l 3 -s 128 -b 2000 -lr 0.001 \
    --use_reward_to_go --use_baseline --gae_lambda $lam \
    --exp_name lunar_lander_lambda${lam}
done
```

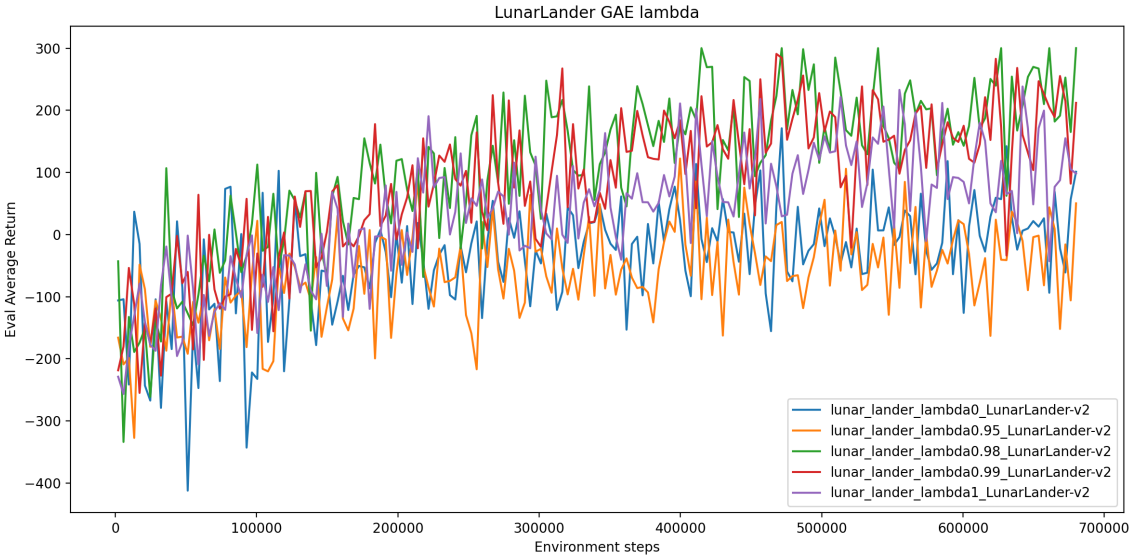


图 5: LunarLander-v2 中不同 λ 的 GAE 学习曲线。

4.4.2 结果分析

LunarLander 的曲线明显比 CartPole 和 HalfCheetah 更抖，这是正常现象。原因是 LunarLander 的奖励更稀疏，policy gradient 本身方差也更大。

从图 5 可以看到，$\lambda = 0$ 的曲线不稳定，早期甚至出现很低的回报。$\lambda = 0.95$ 整体表现一般，很多时候仍在 0 附近或负值。$\lambda = 0.98$、$\lambda = 0.99$ 和 $\lambda = 1$ 后期表现较好，多次达到 180 或 200 以上。

4.4.3 问题回答

λ 控制 GAE 中 bias 与 variance 的折中：

- $\lambda = 0$ 对应 one-step TD advantage estimate，方差低但偏差高；
- $\lambda = 1$ 接近 Monte Carlo advantage estimate，偏差低但方差高；
- 中间值在二者之间折中。

本实验中，中高 λ 的表现更好，说明 LunarLander 这个任务需要更长 horizon 的优势估计，过短的 TD 估计偏差过大。

4.5 Experiment 4：InvertedPendulum 超参数调节与 sample efficiency

4.5.1 运行命令

默认设置：

Listing 16: InvertedPendulum 默认设置，5 个 seed

```
for seed in $(seq 1 5); do
  python cs285/scripts/run_hw2.py \
    --env_name InvertedPendulum-v4 -n 100 \
    --exp_name pendulum_default_s$seed \
    -rtg --use_baseline -na \
    --batch_size 5000 \
    --seed $seed
done
```

调参设置：

Listing 17: InvertedPendulum tuned 设置，5 个 seed

```
for seed in $(seq 1 5); do
  python cs285/scripts/run_hw2.py \
    --env_name InvertedPendulum-v4 -n 100 \
    --exp_name pendulum_tuned_b1000_lr5e3_gae095_s$seed \
    -rtg --use_baseline -na \
    --gae_lambda 0.95 \
    --discount 0.99 \
    --batch_size 1000 \
    --learning_rate 0.005 \
    --n_layers 2 \
    --layer_size 64 \
    --baseline_gradient_steps 5 \
    --seed $seed
done
```

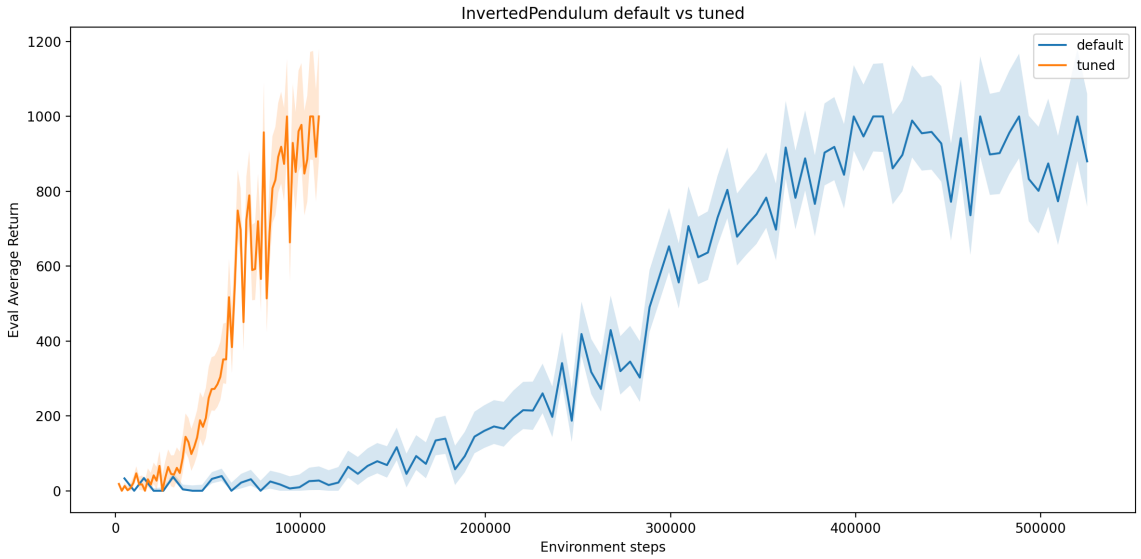


图 6: InvertedPendulum-v4：默认设置与 tuned 设置的平均学习曲线。阴影表示不同 seed 的波动。

4.5.2 超参数表

参数	Default	Tuned
Environment	InvertedPendulum-v4	InvertedPendulum-v4
Batch size	5000	1000
Learning rate	default	0.005
Reward-to-go	yes	yes
Baseline	yes	yes
Advantage normalization	yes	yes
GAE lambda	none	0.95
Discount	default	0.99
Hidden layers	default	2
Hidden size	default	64
Baseline gradient steps	default	5
Seeds	1–5	1–5

4.5.3 结果分析

从图 6 可以看出，tuned 设置明显提升了 sample efficiency。默认设置最终也能接近最高分 1000，但大约需要 3×10^5 到 3.5×10^5 environment steps 后才逐渐接近最高分；tuned 设置大约在 8×10^4 到 1×10^5 steps 附近就能接近 1000。

主要原因是 tuned 设置使用更小的 batch size。默认 batch size 为 5000，100 次 iteration 大约消耗 50 万环境步；tuned batch size 为 1000，100 次 iteration 只消耗约 10 万环境步。同样的 environment steps 下，tuned 设置能进行更多次 policy update，因此样本效率更高。

4.6 Extra Credit：Humanoid

Humanoid-v4 是 extra credit，训练时间很长，作业说明中也提示可能需要 10 小时以上。
结果在"hw2\75轮.gif"

5 Section 8 理论题完整答案

5.1 题目设定

考虑一个无限时域 MDP：在状态 s_1，如果选择 a_1，agent 得到 reward 1 并留在 s_1；如果选择 a_2，agent 得到 reward 0 并终止。策略为：

$$\pi_\theta(a_1|s_1) = \theta, \qquad \pi_\theta(a_2|s_1) = 1-\theta. \tag{22}$$

令 K 表示终止前连续选择 a_1 的次数，则：

$$P(K=k) = (1-\theta)\theta^k, \qquad R(\tau) = k. \tag{23}$$

5.2 8.1 Applying policy gradients

5.2.1 (a) 用 policy gradient 求 $\nabla_\theta J(\theta)$

轨迹概率为：

$$P(K=k) = (1-\theta)\theta^k. \tag{24}$$

因此：

$$\log P(K=k) = k\log\theta + \log(1-\theta). \tag{25}$$

对 θ 求导：

$$\nabla_\theta \log P(K=k) = \frac{k}{\theta} - \frac{1}{1-\theta}. \tag{26}$$

Policy gradient estimator 为：

$$g(k) = R(\tau)\nabla_\theta \log P(K=k) = k\left(\frac{k}{\theta} - \frac{1}{1-\theta}\right). \tag{27}$$

所以：

$$\nabla_\theta J(\theta) = \mathbb{E}[g(K)] \tag{28}$$

$$= \frac{1}{\theta}\mathbb{E}[K^2] - \frac{1}{1-\theta}\mathbb{E}[K]. \tag{29}$$

对几何分布 $P(K=k) = (1-\theta)\theta^k$，有：

$$\mathbb{E}[K] = \frac{\theta}{1-\theta}, \qquad \mathbb{E}[K^2] = \frac{\theta(1+\theta)}{(1-\theta)^2}. \tag{30}$$

代入得：

$$\nabla_\theta J(\theta) = \frac{1}{\theta}\cdot\frac{\theta(1+\theta)}{(1-\theta)^2} - \frac{1}{1-\theta}\cdot\frac{\theta}{1-\theta} \tag{31}$$

$$= \frac{1+\theta}{(1-\theta)^2} - \frac{\theta}{(1-\theta)^2} \tag{32}$$

$$= \boxed{\frac{1}{(1-\theta)^2}}. \tag{33}$$

5.2.2 (b) 直接计算 expected return 并验证

直接计算 expected return：

$$J(\theta) = \mathbb{E}[R(\tau)] \tag{34}$$
$$= \mathbb{E}[K] \tag{35}$$
$$= \sum_{k=0}^{\infty} k(1-\theta)\theta^k \tag{36}$$
$$= \frac{\theta}{1-\theta}. \tag{37}$$

求导：

$$\frac{d}{d\theta}\frac{\theta}{1-\theta} = \frac{1}{(1-\theta)^2}. \tag{38}$$

这与 policy gradient 得到的结果一致。

5.3 8.2 原始 policy gradient 的方差

原始 estimator 为：

$$g(K) = K\left(\frac{K}{\theta} - \frac{1}{1-\theta}\right). \tag{39}$$

因此：

$$\mathrm{Var}[g(K)] = \mathbb{E}[g(K)^2] - \mathbb{E}[g(K)]^2. \tag{40}$$

经过代数化简，得到：

$$\boxed{\mathrm{Var}[g(K)] = \frac{4\theta^2 + 8\theta + 1}{\theta(1-\theta)^4}}. \tag{41}$$

性质：

- 当 $\theta \to 0$ 时，分母中的 θ 使方差趋于无穷；
- 当 $\theta \to 1$ 时，分母中的 $(1-\theta)^4$ 使方差趋于无穷；
- 在区间 $(0,1)$ 内方差有最小值，约在 $\theta \approx 0.1099$ 附近。

直观解释：θ 很小时，成功选择 a_1 很少见，估计器偶尔出现大值；θ 很大时，轨迹会非常长，return 和梯度都可能很大，因此方差也会爆炸。

5.4 8.3 Return-to-go 作为 advantage estimator

5.4.1 (a) 写出 modified policy gradient 并说明 unbiased

Return-to-go estimator 为：

$$g_{\mathrm{RTG}}(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \left(\sum_{t'=t}^{T-1} r_{t'}\right). \tag{42}$$

在本题中，如果连续选择 $K = k$ 次 a_1，则每一步的 reward-to-go 分别为：

$$k, k-1, k-2, \ldots, 1. \tag{43}$$

每次选择 a_1 的 score function 是：

$$\nabla_\theta \log \pi_\theta(a_1|s_1) = \frac{1}{\theta}. \tag{44}$$

终止动作 a_2 的 reward-to-go 为 0，所以不贡献梯度。因此：

$$g_{\text{RTG}}(K) = \frac{1}{\theta}\sum_{j=1}^{K} j \tag{45}$$

$$= \frac{K(K+1)}{2\theta}. \tag{46}$$

取期望：

$$\mathbb{E}[g_{\text{RTG}}(K)] = \sum_{k=0}^{\infty}(1-\theta)\theta^k \frac{k(k+1)}{2\theta} \tag{47}$$

$$= \frac{1}{(1-\theta)^2}. \tag{48}$$

这与真实梯度一致，因此 return-to-go estimator 是 unbiased。

5.4.2 (b) Return-to-go 方差

Return-to-go estimator 为：

$$g_{\text{RTG}}(K) = \frac{K(K+1)}{2\theta}. \tag{49}$$

方差为：

$$\boxed{\text{Var}[g_{\text{RTG}}(K)] = \frac{\theta^2+3\theta+1}{\theta(1-\theta)^4}}. \tag{50}$$

与原始 estimator 相比：

$$\text{Var}[g(K)] - \text{Var}[g_{\text{RTG}}(K)] = \frac{4\theta^2+8\theta+1}{\theta(1-\theta)^4} - \frac{\theta^2+3\theta+1}{\theta(1-\theta)^4} \tag{51}$$

$$= \frac{3\theta^2+5\theta}{\theta(1-\theta)^4} \tag{52}$$

$$= \frac{3\theta+5}{(1-\theta)^4} > 0. \tag{53}$$

所以：

$$\boxed{\text{Var}[g_{\text{RTG}}] < \text{Var}[g]}. \tag{54}$$

这说明 reward-to-go 严格降低了方差。

5.5 8.4 Importance sampling policy gradients

考虑有限 horizon H 的 sparse reward MDP。要到达 s_H，需要连续选择 a_1。令：

$$L = H - 1. \tag{55}$$

目标策略为 π_θ，行为策略为 $\pi_{\theta'}$。

5.5.1 (a) 写出 importance sampling policy gradient

Off-policy policy gradient 为：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_{\theta'}}\left[\frac{\pi_\theta(\tau)}{\pi_{\theta'}(\tau)}\nabla_\theta \log \pi_\theta(\tau) R(\tau)\right]. \tag{56}$$

只有成功轨迹 reward 为 1，失败轨迹 reward 为 0。成功轨迹概率为：

$$\pi_\theta(\tau_{\text{success}}) = \theta^L, \qquad \pi_{\theta'}(\tau_{\text{success}}) = (\theta')^L. \tag{57}$$

所以 importance weight 为：

$$w(\tau_{\text{success}}) = \left(\frac{\theta}{\theta'}\right)^L. \tag{58}$$

成功轨迹上的 score function 为：

$$\nabla_\theta \log \pi_\theta(\tau_{\text{success}}) = \nabla_\theta L \log \theta = \frac{L}{\theta}. \tag{59}$$

因此成功轨迹上的 estimator 为：

$$g_{\text{IS}} = \left(\frac{\theta}{\theta'}\right)^L \frac{L}{\theta} = \frac{L\theta^{L-1}}{(\theta')^L}. \tag{60}$$

失败轨迹 estimator 为 0。

5.5.2 (b) 计算方差并讨论 H 变大时的影响

在行为策略 $\pi_{\theta'}$ 下，成功概率为 $(\theta')^L$。因此：

$$\mathbb{E}[g_{\text{IS}}] = (\theta')^L \frac{L\theta^{L-1}}{(\theta')^L} = L\theta^{L-1}. \tag{61}$$

这正是 $\nabla_\theta \theta^L$，所以 estimator 是 unbiased。

二阶矩为：

$$\mathbb{E}[g_{\text{IS}}^2] = (\theta')^L \left(\frac{L\theta^{L-1}}{(\theta')^L}\right)^2 = \frac{L^2\theta^{2L-2}}{(\theta')^L}. \tag{62}$$

因此方差为：

$$\begin{aligned}
\text{Var}[g_{\text{IS}}] &= \mathbb{E}[g_{\text{IS}}^2] - \mathbb{E}[g_{\text{IS}}]^2 && (63)\\
&= \frac{L^2\theta^{2L-2}}{(\theta')^L} - L^2\theta^{2L-2} && (64)\\
&= \boxed{L^2\theta^{2L-2}\left(\frac{1}{(\theta')^L} - 1\right)}. && (65)
\end{aligned}$$

因为 $L = H - 1$，当 H 很大时，$(\theta')^L$ 会非常小，$1/(\theta')^L$ 会非常大。因此 off-policy importance sampling policy gradient 的方差会随 horizon 增大而急剧变大，甚至呈指数级爆炸。

6 Survey 时间估计

以下时间是本次完成作业的估计。

部分	写代码时间（分钟）	等待实验时间（分钟）
Policy Gradients	60	40
Neural Network Baseline	45	50
Generalized Advantage Estimation	40	120
Hyperparameters and Sample Efficiency	35	90
Humanoid	0	0
Analysis 1.1 Applying policy gradients	35	0
Analysis 1.2 Policy gradient variance	50	0
Analysis 1.3 Return-to-go	45	0
Analysis 1.4 Importance sampling	45	0

7 最终提交说明

7.1 报告 PDF 应包含什么

最终 PDF 应包含：

- Experiment 1–4 的所有图；
- 每个实验的命令行参数；
- 每个实验的问题回答；
- Section 8 理论题推导；
- Survey 时间估计。

7.2 代码与日志打包

建议使用如下结构：

Listing 18: submit.zip 结构

```
submit.zip
  run_logs/
  cs285/
  README.md
```

打包命令示例：

Listing 19: 打包命令示例

```
rm -rf submit
mkdir -p submit/run_logs

cp -r data/q2_pg_cartpole* submit/run_logs/
cp -r data/q2_pg_cheetah* submit/run_logs/
cp -r data/q2_pg_lunar_lander* submit/run_logs/
cp -r data/q2_pg_pendulum* submit/run_logs/

cp -r cs285 submit/cs285
rm -rf submit/cs285/data

find submit -type f \( -name "*.mp4" -o -name "*.gif" -o -name "*.webm" \) -delete

cat > submit/README.md <<'EOF'
This submission contains completed CS285 HW2 code and experiment logs.
Figures use Train_EnvstepsSoFar as the x-axis.
EOF

zip -vr submit.zip submit -x "*.DS_Store"
ls -lh submit.zip
```

8 自己的思考与总结

8.1 Policy Gradient 的根本困难是方差

本作业最核心的体会是：Policy Gradient 的公式看起来很简单，但实际训练时最大的困难不是公式推导，而是**方差太大**。同一个环境中，采样出来的 trajectory 可能差异很大，导致梯度估计不稳定。因此很多技巧其实都围绕一个目标：让梯度估计更稳定。

8.2 Reward-to-go 是最自然的 credit assignment 修正

Reward-to-go 的思想非常直观：一个动作不应该为它之前的奖励负责。这是因果性在 policy gradient 中的体现。它不会改变梯度期望，但能降低方差，因此几乎是最基本、最应该使用的改进。

8.3 Baseline 不是越复杂越好，而是要训练充分

HalfCheetah 实验说明，使用 baseline 能明显提升表现，但 baseline 也不是“加上就自动好”。如果 baseline gradient steps 太少，value function 学不准，advantage 仍然噪声很大。也就是说，actor 和 critic 之间存在配合关系：critic 太弱，actor 的学习也会受影响。

8.4 GAE 的 λ 没有绝对最优

GAE 的 λ 是 bias-variance tradeoff。$\lambda = 0$ 偏向短视，$\lambda = 1$ 偏向完整 Monte Carlo。不同任务中最合适的 λ 不同。LunarLander 中较高的 λ 表现更好，说明它需要更长 horizon 的 return 信息。

8.5 Sample efficiency 不能只看 iteration

InvertedPendulum 实验说明，比较算法时不能只看 iteration。因为 batch size 不同，每个 iteration 消耗的 environment steps 不同。作业要求横坐标使用 `Train_EnvstepsSoFar` 是非常合理的。真正关心 sample efficiency 时，应当看用多少环境交互步数达到目标性能。

8.6 最终一句话总结

Policy Gradient 的基本公式是“好动作多做，坏动作少做”；reward-to-go、normalization、baseline 和 GAE 都是在让“好坏判断”更准确、更稳定。